Mini project report on

Operating System

Submitted in partial fulfilment of the requirements for the award of degree of

Bachelor of Technology

in

Computer Science & Engineering

UE24CS242B - Operating Systems

Submitted by:

Heemadhawala R PES2UG24CS190

Hemavathi E PES2UG24CS193

under the guidance of

Prof. Shilpa S

Assistant Professor

PES University

Jan- May 2026

DEPARTMENT OF COMPUTER SCIENCE AND ENGINEERING

FACULTY OF ENGINEERING

PES UNIVERSITY

(Established under Karnataka Act No. 16 of 2013)

Electronic City, Hosur Road, Bengaluru – 560 100, Karnataka, India

PES UNIVERSITY

(Established under Karnataka Act No. 16 of 2013)

Electronic City, Hosur Road, Bengaluru – 560 100, Karnataka, India

CERTIFICATE

This is to certify that the mini project entitled

Operating Systems

is a bonafide work carried out by

Heemadhawala R	PES2UG24CS190
Hemavathi E	PES2UG24CS193

In partial fulfilment for the completion of forth semester OS Project (UE24CS242B) in the Program of Study -Bachelor of Technology in Computer Science and Engineering under rules and regulations of PES University, Bengaluru during the period Jan 2026 – May 2026. It is certified that all corrections / suggestions indicated for internal assessment have been incorporated in the report. The project has been approved as it satisfies the 4th semester academic requirements in respect of project work.

Signature

Prof. Shilpa S

Assistant Professor

DECLARATION

We hereby declare that the DBMS Project entitled Operating System has been carried out by us under the guidance of Prof. Shilpa S, Assistant Professor and submitted in partial fulfilment of the course requirements for the award of degree of Bachelor of Technology in Computer Science and Engineering of PES University, Bengaluru during the academic semester Jan – May 2026.

Heemadhawala R	PES2UG24CS190
Hemavathi E	PES2UG24CS193

ACKNOWLEDGEMENT

I would like to express my gratitude to Prof. Shilpa S, Department of Computer Science and Engineering, PES University, for her continuous guidance, assistance, and encouragement throughout the development of this UE24CS242B - Operating Systems Project.

I take this opportunity to thank Dr. Sandesh B J, C, Professor, ChairPerson, Department of Computer Science and Engineering, PES University, for all the knowledge and support I have received from the department.

I am deeply grateful to Prof. Jawahar Doreswamy, Chancellor – PES University, Dr. Suryaprasad J, Vice-Chancellor, PES University for providing to me various opportunities and enlightenment every step of the way. Finally, this DBMS Project could not have been completed without the continual support and encouragement I have received from my family and friends.

ABSTRACT

This project presents the implementation of a multi-container runtime system in C that supports concurrent container execution under a single supervisor process. The system enables creation, monitoring, and control of containers using a command-line interface, while ensuring process and filesystem isolation.

A bounded-buffer logging mechanism is implemented to reliably capture output from multiple containers without data loss. The system also integrates a kernel module to monitor memory usage and enforce soft and hard limits on container processes.

Experimental results show that multiple containers can run simultaneously with proper isolation, logging is handled correctly under concurrent conditions, and memory limits are enforced as expected. Scheduling experiments further demonstrate how process priority affects CPU allocation and execution behavior.

TABLE OF CONTENTS

Chapter No.	Title	Page No.
1.	INTRODUCTION	9
2.	PROBLEM DEFINITION	10
3.	METHODOLOGY	11
4.	CODE	12
5.	RESULT AND DISCURSION	29
6.	OUTPUTS	31
7.	GITHUB LINK	36
REFERENCES/BIBLIOGRAPHY		37
APPENDIX A DEFINITIONS, ACRONYMS AND ABBREVIATIONS		38

LIST OF FIGURES

FIG NO	Title	Page number
6.1	Multi-Container Supervision	31
6.2	Meta data tracking	31
6.3	Bounded buffer logging	32
6.4	CLI and IPC	32
6.5	Soft-Limit Warning	33
6.6	Hard-Limit Enforcement	33
6.7	Scheduling Experiment	34
6.8	Clean Teardown	35

CHAPTER 1:- INTRODUCTION

Operating systems manage the overall functioning of the computer by managing the CPU, memory, and processing and running multiple applications seamlessly. A significant challenge with most modern applications is achieving process isolation without a significant drop in performance.

Virtual machines are the standard for strong isolation but with the drawback of complete isolation and significant resource consumption due to the multiple instances of the virtual machines. Containers are a more efficient solution. Also, because they share the host operating system, they are more efficient in their resource consumption.

Most container systems provide a simplified version of the container system without providing the user with the complete system details of the complexities of the system. Most of the time, most users see the advantages, but few know the complete details and the interaction with the operating systems. As a system, the operating system utilizes processes, process separation, process isolation, and contained systems.

Most of the time, container systems provide the user with a complete abstraction of the simplified system, and therefore the user is unaware of the complexities that are involved in building the complete system. Most users simply see the advantages, but that fails to recognize the complete system. In this project, most of the operating systems' processes, and their usage in isolation, are utilized. Also, in this project, most of the complex systems that provide the complete system functioning are utilized to their fullest extent.

With most of the modern operating systems, this project is able to look deeply into memory control, inter-process systems, isolation, synchronization, process control, and the complex systems that are involved in scheduling operating systems. Most of the time, this is simply taken for granted, but the failure to understand modern systems is usually reserved for most systems. Most systems would not need to go to the degree of complete isolation that this system would be able to provide. Most of the time, that complete isolation would be reserved for modern systems.

CHAPTER 2:- PROBLEM DEFINITION

2.1 Problem Statement

System design problems become more challenging with the increasing complexity of the isolation of processes sharing the same kernel system.

A container is more lightweight than a virtual machine, as it offers process isolation with just one kernel. Many container platforms, however, abstract inner workings, which makes it difficult to understand the layers of isolation, process, and resource control.

A problem that is being sought in this case is creating a multi-container runtime system that is lightweight, and able to run several containers via a single supervisor process. The system must provide sufficient isolation, a user interaction functionality, and handle control of resource utilization.

Furthermore, it has to provide the functionality to control and monitor the memory.

2.2 Objectives

Main goals for the project include:-

*To provide execution of several containers simultaneously under a single supervisor process

*To maintain protection of isolation of processes and file systems between different containers

*To design and implement a command line interface to enable control of the providing life span of containers

*To ensure proper control and monitoring together with providing systematic and reliable means to capture and save output of containers

*To enable and study influence of process scheduling to monitor and control memory consumed by container processes

CHAPTER 3:- METHODOLOGY

3.1 System Architecture

The system is built as a single program, called the engine, that operates in two modes.

Supervisor mode runs continuously and manages all containers.

CLI mode sends commands like start, stop, ps, and logs to the supervisor and then exits.

The supervisor is the central controller. It creates containers, tracks their state, and handles logging. The CLI talks to the supervisor using a UNIX domain socket by sending structured requests and receiving responses.

The kernel module loads using insmod and registers a character device (/dev/container_monitor) with device_create.

The module has ioctl handlers to support operations such as container and process registration, monitoring, and control.

A userspace engine program interacts with the kernel module through open() and ioctl() system calls.

The engine processes supervisor commands like ps and stop, then forwards structured requests to the kernel.

The kernel handles these requests and keeps track of the runtime state of monitored processes and containers.

A SIGCHLD handler in userspace reaps terminated child processes to prevent zombie processes.

The system allows communication between kernel space and userspace for dynamic process and container monitoring.

CHAPTER 4:- CODE

engine.c - Supervised Multi-Container Runtime (User Space)

```c
#define _GNU_SOURCE
#include <errno.h>
#include <fcntl.h>
#include <limits.h>
#include <pthread.h>
#include <sched.h>
#include <signal.h>
#include <stdio.h>
#include <stdlib.h>
#include <string.h>
#include <sys/ioctl.h>
#include <sys/mount.h>
#include <sys/socket.h>
#include <sys/stat.h>
#include <sys/types.h>
#include <sys/un.h>
#include <sys/wait.h>
#include <time.h>
#include <unistd.h>
#include "monitor_ioctl.h"
#define STACK_SIZE (1024 * 1024)
#define CONTAINER_ID_LEN 32
#define CONTROL_PATH "/tmp/mini_runtime.sock"
#define LOG_DIR "logs"
#define CONTROL_MESSAGE_LEN 256
#define CHILD_COMMAND_LEN 256
#define LOG_CHUNK_SIZE 4096
#define LOG_BUFFER_CAPACITY 16
#define DEFAULT_SOFT_LIMIT (40UL << 20)
#define DEFAULT_HARD_LIMIT (64UL << 20)
typedef enum {
   CMD_SUPERVISOR = 0,
   CMD_START,
   CMD_RUN,
   CMD_PS,
   CMD_LOGS,
   CMD_STOP
} command_kind_t;
typedef enum {
   CONTAINER_STARTING = 0,
   CONTAINER_RUNNING,
   CONTAINER_STOPPED,
   CONTAINER_KILLED,
   CONTAINER_EXITED
} container_state_t;
typedef struct container_record {
   char id[CONTAINER_ID_LEN];
   pid_t host_pid;
   time_t started_at;
   container_state_t state;
   unsigned long soft_limit_bytes;
   unsigned long hard_limit_bytes;
   int exit_code;
   int exit_signal;
   char log_path[PATH_MAX];
   struct container_record *next;
```

```
} container_record_t;
typedef struct {
    char container_id[CONTAINER_ID_LEN];
    size_t length;
    char data[LOG_CHUNK_SIZE];
} log_item_t;
typedef struct {
    log_item_t items[LOG_BUFFER_CAPACITY];
    size_t head;
    size_t tail;
    size_t count;
    int shutting_down;
    pthread_mutex_t mutex;
    pthread_cond_t not_empty;
    pthread_cond_t not_full;
} bounded_buffer_t;
typedef struct {
    command_kind_t kind;
    char container_id[CONTAINER_ID_LEN];
    char rootfs[PATH_MAX];
    char command[CHILD_COMMAND_LEN];
    unsigned long soft_limit_bytes;
    unsigned long hard_limit_bytes;
    int nice_value;
} control_request_t;
typedef struct {
    int status;
    char message[CONTROL_MESSAGE_LEN];
} control_response_t;
typedef struct {
    char id[CONTAINER_ID_LEN];
    char rootfs[PATH_MAX];
    char command[CHILD_COMMAND_LEN];
    int nice_value;
    int log_write_fd;
} child_config_t;
typedef struct {
    int server_fd;
    int monitor_fd;
    int should_stop;
    pthread_t logger_thread;
    bounded_buffer_t log_buffer;
    pthread_mutex_t metadata_lock;
    container_record_t *containers;
} supervisor_ctx_t;
typedef struct{
    int fd;
    supervisor_ctx_t *ctx;
    char id[CONTAINER_ID_LEN];
} producer_arg_t;
static void usage(const char *prog)
{
    fprintf(stderr,
            "Usage:\n"
            "  %s supervisor <base-rootfs>\n"
            "  %s start <id> <container-rootfs> <command> [--soft-mib N] [--hard-mib N] [--nice N]\n"
            "  %s run <id> <container-rootfs> <command> [--soft-mib N] [--hard-mib N] [--nice N]\n"
            "  %s ps\n"
            "  %s logs <id>\n"
            "  %s stop <id>\n",
            prog, prog, prog, prog, prog, prog);
}
static int parse_mib_flag(const char *flag,
```

```c
                const char *value,
                unsigned long *target_bytes)
{
    char *end = NULL;
    unsigned long mib;
    errno = 0;
    mib = strtoul(value, &end, 10);
    if (errno != 0 || end == value || *end != '\0') {
        fprintf(stderr, "Invalid value for %s: %s\n", flag, value);
        return -1;
    }
    if (mib > ULONG_MAX / (1UL << 20)) {
        fprintf(stderr, "Value for %s is too large: %s\n", flag, value);
        return -1;
    }
    *target_bytes = mib * (1UL << 20);
    return 0;
}
static int parse_optional_flags(control_request_t *req,
                int argc,
                char *argv[],
                int start_index)
{
    int i;
    for (i = start_index; i < argc; i += 2) {
        char *end = NULL;
        long nice_value;
        if (i + 1 >= argc) {
            fprintf(stderr, "Missing value for option: %s\n", argv[i]);
            return -1;
        }
        if (strcmp(argv[i], "--soft-mib") == 0) {
            if (parse_mib_flag("--soft-mib", argv[i + 1], &req->soft_limit_bytes) != 0)
                return -1;
            continue;
        }
        if (strcmp(argv[i], "--hard-mib") == 0) {
            if (parse_mib_flag("--hard-mib", argv[i + 1], &req->hard_limit_bytes) != 0)
                return -1;
            continue;
        }
        if (strcmp(argv[i], "--nice") == 0) {
            errno = 0;
            nice_value = strtol(argv[i + 1], &end, 10);
            if (errno != 0 || end == argv[i + 1] || *end != '\0' ||
                nice_value < -20 || nice_value > 19) {
                fprintf(stderr,
                        "Invalid value for --nice (expected -20..19): %s\n",
                        argv[i + 1]);
                return -1;
            }
            req->nice_value = (int)nice_value;
            continue;
        }
        fprintf(stderr, "Unknown option: %s\n", argv[i]);
        return -1;
    }
    if (req->soft_limit_bytes > req->hard_limit_bytes) {
        fprintf(stderr, "Invalid limits: soft limit cannot exceed hard limit\n");
        return -1;
    }
    return 0;
}
```

```c
static const char *state_to_string(container_state_t state)
{
    switch (state) {
    case CONTAINER_STARTING:
        return "starting";
    case CONTAINER_RUNNING:
        return "running";
    case CONTAINER_STOPPED:
        return "stopped";
    case CONTAINER_KILLED:
        return "killed";
    case CONTAINER_EXITED:
        return "exited";
    default:
        return "unknown";
    }
}
static int bounded_buffer_init(bounded_buffer_t *buffer)
{
    int rc;
    memset(buffer, 0, sizeof(*buffer));
    rc = pthread_mutex_init(&buffer->mutex, NULL);
    if (rc != 0)
        return rc;
    rc = pthread_cond_init(&buffer->not_empty, NULL);
    if (rc != 0) {
        pthread_mutex_destroy(&buffer->mutex);
        return rc;
    }
    rc = pthread_cond_init(&buffer->not_full, NULL);
    if (rc != 0) {
        pthread_cond_destroy(&buffer->not_empty);
        pthread_mutex_destroy(&buffer->mutex);
        return rc;
    }
    return 0;
}
static void bounded_buffer_destroy(bounded_buffer_t *buffer)
{
    pthread_cond_destroy(&buffer->not_full);
    pthread_cond_destroy(&buffer->not_empty);
    pthread_mutex_destroy(&buffer->mutex);
}
static void bounded_buffer_begin_shutdown(bounded_buffer_t *buffer)
{
    pthread_mutex_lock(&buffer->mutex);
    buffer->shutting_down = 1;
    pthread_cond_broadcast(&buffer->not_empty);
    pthread_cond_broadcast(&buffer->not_full);
    pthread_mutex_unlock(&buffer->mutex);
}
int bounded_buffer_push(bounded_buffer_t *buffer, const log_item_t *item)
{
    pthread_mutex_lock(&buffer->mutex);
    while(buffer->count == LOG_BUFFER_CAPACITY && !buffer->shutting_down){
            pthread_cond_wait(&buffer->not_full, &buffer->mutex);
    }
    if(buffer->shutting_down){
            pthread_mutex_unlock(&buffer->mutex);
            return -1;
    }
    buffer->items[buffer->tail] = *item;
    buffer->tail = (buffer->tail+1)%LOG_BUFFER_CAPACITY;
```

```
    buffer->count++;

    pthread_cond_signal(&buffer->not_empty);
    pthread_mutex_unlock(&buffer->mutex);
    return 0;
}
int bounded_buffer_pop(bounded_buffer_t *buffer, log_item_t *item)
{
    pthread_mutex_lock(&buffer->mutex);

    while(buffer->count == 0 && !buffer->shutting_down){
            pthread_cond_wait(&buffer->not_empty, &buffer->mutex);
    }
    if(buffer->count==0){
            pthread_mutex_unlock(&buffer->mutex);
            return -1;
    }
    *item = buffer->items[buffer->head];
    buffer->head = (buffer->head+1)%LOG_BUFFER_CAPACITY;
    buffer->count--;
    pthread_cond_signal(&buffer->not_full);
    pthread_mutex_unlock(&buffer->mutex);
    return 0;
}
void *logging_thread(void *arg){
            supervisor_ctx_t *ctx = (supervisor_ctx_t *)arg;
            log_item_t item;
            char log_path[PATH_MAX];
            int fd;
            mkdir(LOG_DIR, 0755);
            while(bounded_buffer_pop(&ctx->log_buffer, &item) == 0){
                snprintf(log_path, sizeof(log_path), "%s/%s.log",
                                LOG_DIR, item.container_id);

                fd = open(log_path, O_WRONLY | O_CREAT | O_APPEND, 0644);
                if(fd<0) continue;
                write(fd, item.data, item.length);
                close(fd);
            }
            return NULL;
}
int child_fn(void *arg)
{
    child_config_t *cfg = (child_config_t *) arg;
    dup2(cfg->log_write_fd, STDOUT_FILENO);
    dup2(cfg->log_write_fd, STDERR_FILENO);
    close(cfg->log_write_fd);
    sethostname(cfg->id, strlen(cfg->id));

    char proc_path[PATH_MAX];
    snprintf(proc_path, sizeof(proc_path), "%s/proc", cfg->rootfs);
    mount("proc", proc_path, "proc", 0, NULL);
    if(chroot(cfg->rootfs)!=0){
            perror("chroot");
            return 1;
    }
    chdir("/");
    if(cfg->nice_value != 0){
            nice(cfg->nice_value);
    }
    char *args[] = { cfg->command, NULL };
    execv(cfg->command, args);
    perror("execv");
```

```
    return 1;
}
int register_with_monitor(int monitor_fd,
                          const char *container_id,
                          pid_t host_pid,
                          unsigned long soft_limit_bytes,
                          unsigned long hard_limit_bytes)
{
    struct monitor_request req;
    memset(&req, 0, sizeof(req));
    req.pid = host_pid;
    req.soft_limit_bytes = soft_limit_bytes;
    req.hard_limit_bytes = hard_limit_bytes;
    strncpy(req.container_id, container_id, sizeof(req.container_id) - 1);
    if (ioctl(monitor_fd, MONITOR_REGISTER, &req) < 0)
        return -1;
    return 0;
}
int unregister_from_monitor(int monitor_fd, const char *container_id, pid_t host_pid)
{
    struct monitor_request req;
    memset(&req, 0, sizeof(req));
    req.pid = host_pid;
    strncpy(req.container_id, container_id, sizeof(req.container_id) - 1);
    if (ioctl(monitor_fd, MONITOR_UNREGISTER, &req) < 0)
        return -1;
    return 0;
}
static void reap_children(supervisor_ctx_t *ctx){
        int status;
        pid_t pid;
        while((pid = waitpid(-1, &status, WNOHANG))>0){
            pthread_mutex_lock(&ctx->metadata_lock);
            container_record_t *c = ctx->containers;
            while(c){
                    if(c->host_pid == pid){
                            if(WIFEXITED(status)){
                                    c->exit_code = WEXITSTATUS(status);
                                    c->state = CONTAINER_EXITED;
                            } else if(WIFSIGNALED(status)){
                                    c->exit_signal = WTERMSIG(status);
                                    if(c->state != CONTAINER_STOPPED){
                                            c->state = CONTAINER_KILLED;
                                    }
                            }
                            break;
                    }
                    c = c->next;
            }
            pthread_mutex_unlock(&ctx->metadata_lock);
        }
}
static supervisor_ctx_t *g_ctx = NULL;
static void sigchld_handler(int sig){
    (void)sig;
    if(g_ctx) reap_children(g_ctx);
}
static void sigterm_handler(int sig){
    (void)sig;
    if(g_ctx){
        g_ctx->should_stop = 1;
        close(g_ctx->server_fd);
    }
```

```
}
static void *producer_thread(void *arg){
    producer_arg_t *parg = (producer_arg_t *)arg;
    log_item_t item;
    ssize_t n;
    while((n = read(parg->fd, item.data, LOG_CHUNK_SIZE))>0){
        item.length = n;
        strncpy(item.container_id, parg->id, CONTAINER_ID_LEN-1);
        bounded_buffer_push(&parg->ctx->log_buffer, &item);
    }
    close(parg->fd);
    free(parg);
    return NULL;
}
static void handle_start(supervisor_ctx_t *ctx, control_request_t *req, control_response_t *resp, int client_fd){
    int pipefd[2];
    if(pipe(pipefd)<0){
        resp->status = -1;
        snprintf(resp->message, sizeof(resp->message), "pipe failed");
        write(client_fd, resp, sizeof(*resp));
        return;
    }
    char *stack = malloc(STACK_SIZE);
    if(!stack){
        resp->status = -1;
        snprintf(resp->message, sizeof(resp->message), "malloc failed");
        write(client_fd, resp, sizeof(*resp));
        return;
    }
    child_config_t *cfg = malloc(sizeof(child_config_t));
    strncpy(cfg->id, req->container_id, sizeof(cfg->id)-1);
    strncpy(cfg->rootfs, req->rootfs, sizeof(cfg->rootfs)-1);
    strncpy(cfg->command, req->command, sizeof(cfg->command)-1);
    cfg->nice_value = req->nice_value;
    cfg->log_write_fd = pipefd[1];
    pid_t pid = clone(child_fn, stack + STACK_SIZE, CLONE_NEWPID | CLONE_NEWUTS | CLONE_NEWNS | SIGCHLD, cfg);
    close(pipefd[1]);
    free(stack);
    if(pid<0){
        perror("clone");
        close(pipefd[0]);
        free(cfg);
        resp->status = -1;
        snprintf(resp->message, sizeof(resp->message), "clone failed");
        write(client_fd, resp, sizeof(*resp));
        return;
    }
    fprintf(stderr, "Received start for %s, spawned pid %d\n", req->container_id, pid);
    if(ctx->monitor_fd>=0){
        register_with_monitor(ctx->monitor_fd, req->container_id, pid, req->soft_limit_bytes, req->hard_limit_bytes);
    }
    container_record_t *rec = calloc(1, sizeof(container_record_t));
    strncpy(rec->id, req->container_id, sizeof(rec->id)-1);
    rec->host_pid = pid;
    rec->started_at = time(NULL);
    rec->state = CONTAINER_RUNNING;
    rec->soft_limit_bytes = req->soft_limit_bytes;
    rec->hard_limit_bytes = req->hard_limit_bytes;
    snprintf(rec->log_path, sizeof(rec->log_path), "%s/%s.log", LOG_DIR, req->container_id);
    pthread_mutex_lock(&ctx->metadata_lock);
    rec->next = ctx->containers;
    ctx->containers = rec;
    pthread_mutex_unlock(&ctx->metadata_lock);
```

```
    producer_arg_t *parg = malloc(sizeof(producer_arg_t));
    parg->fd = pipefd[0];
    parg->ctx = ctx;
    strncpy(parg->id, req->container_id, sizeof(parg->id)-1);
    pthread_t pthr;
    pthread_create(&pthr, NULL, producer_thread, parg);
    pthread_detach(pthr);
    resp->status = 0;
    snprintf(resp->message, sizeof(resp->message), "started pid %d", pid);
    write(client_fd, resp, sizeof(*resp));

    if(req->kind == CMD_RUN){
        int wstatus;
        waitpid(pid, &wstatus, 0);
        int code = WIFEXITED(wstatus) ? WEXITSTATUS(wstatus) : 128 + WTERMSIG(wstatus);
        snprintf(resp->message, sizeof(resp->message), "exited %d", code);
        write(client_fd, resp, sizeof(*resp));
    }
    free(cfg);
}
static void handle_ps(supervisor_ctx_t *ctx, control_response_t *resp, int client_fd){
    char buf[4096] = {0};
    int offset = 0;
    pthread_mutex_lock(&ctx->metadata_lock);
    container_record_t *c = ctx->containers;
    offset += snprintf(buf + offset, sizeof(buf)-offset,
            "%-16s %-8s %-10s %-8s %-8s\n",
            "ID", "PID", "STATE", "SOFT_MIB", "HARD_MIB");
    while(c && offset < (int)sizeof(buf) - 64){
        offset += snprintf(buf + offset, sizeof(buf) - offset,
                    "%-16s %-8d %-10s %-8lu %-8lu\n",
                    c->id, c->host_pid, state_to_string(c->state),
                    c->soft_limit_bytes >> 20,
                    c->hard_limit_bytes >> 20);
        c = c->next;
    }
    pthread_mutex_unlock(&ctx->metadata_lock);
    write(client_fd, buf, strlen(buf));
    resp->status = 0;
}
static void handle_logs(supervisor_ctx_t *ctx, control_request_t *req, control_response_t *resp, int client_fd){
    char log_path[PATH_MAX];
    snprintf(log_path, sizeof(log_path), "%s/%s.log", LOG_DIR, req->container_id);
    int fd = open(log_path, O_RDONLY);
    if (fd < 0) {
        resp->status = -1;
        snprintf(resp->message, sizeof(resp->message), "no logs for %s", req->container_id);
        write(client_fd, resp, sizeof(*resp));
        return;
    }
    char buf[4096];
    ssize_t n;
    while ((n = read(fd, buf, sizeof(buf))) > 0)
        write(client_fd, buf, n);
    close(fd);
    resp->status = 0;
}
static void handle_stop(supervisor_ctx_t *ctx, control_request_t *req, control_response_t *resp){
    pthread_mutex_lock(&ctx->metadata_lock);
    container_record_t *c = ctx->containers;
    while (c) {
        if (strcmp(c->id, req->container_id) == 0) {
            if (c->state == CONTAINER_RUNNING) {
```

```
            c->state = CONTAINER_STOPPED; // set before kill per spec
            kill(c->host_pid, SIGTERM);
            resp->status = 0;
            snprintf(resp->message, sizeof(resp->message),
                    "sent SIGTERM to %d", c->host_pid);
        } else {
            resp->status = -1;
            snprintf(resp->message, sizeof(resp->message),
                    "container %s not running", req->container_id);
        }
        pthread_mutex_unlock(&ctx->metadata_lock);
        return;
    }
    c = c->next;
  }
  pthread_mutex_unlock(&ctx->metadata_lock);
  resp->status = -1;
  snprintf(resp->message, sizeof(resp->message),
          "container %s not found", req->container_id);
}
static int run_supervisor(const char *rootfs)
{
  supervisor_ctx_t ctx;
  struct sockaddr_un addr;
  int rc;
  (void)rootfs;
  memset(&ctx, 0, sizeof(ctx));
  ctx.server_fd = -1;
  ctx.monitor_fd = -1;
  rc = pthread_mutex_init(&ctx.metadata_lock, NULL);
  if (rc != 0) {
    perror("pthread_mutex_init");
    return 1;
  }
  rc = bounded_buffer_init(&ctx.log_buffer);
  if (rc != 0) {
    perror("bounded_buffer_init");
    pthread_mutex_destroy(&ctx.metadata_lock);
    return 1;
  }
  ctx.monitor_fd = open("/dev/container_monitor", O_RDWR);
  if(ctx.monitor_fd < 0){
        perror("open /dev/container_monitor");
  }
  //todo 2
  unlink(CONTROL_PATH);
  ctx.server_fd = socket(AF_UNIX, SOCK_STREAM, 0);
  if(ctx.server_fd < 0){
    perror("socket"); return 1;
  }
  memset(&addr, 0, sizeof(addr));
  addr.sun_family = AF_UNIX;
  strncpy(addr.sun_path, CONTROL_PATH, sizeof(addr.sun_path)-1);
  if(bind(ctx.server_fd, (struct sockaddr *)&addr, sizeof(addr))<0){
    perror("bind");
        return 1;
  }
  if(listen(ctx.server_fd, 8)<0){
    perror("listen");
        return 1;
  }
  //todo 3
  rc = pthread_create(&ctx.logger_thread, NULL, logging_thread, &ctx);
```

```
if(rc!=0){
   perror("pthread_create");
      return 1;
}
mkdir(LOG_DIR, 0755);
g_ctx = &ctx;
struct sigaction sa;
memset(&sa, 0, sizeof(sa));
sa.sa_handler = sigchld_handler;
sa.sa_flags = SA_RESTART | SA_NOCLDSTOP;
sigaction(SIGCHLD, &sa, NULL);
sa.sa_handler = sigterm_handler;
sa.sa_flags = 0;
sigaction(SIGINT, &sa, NULL);
sigaction(SIGTERM, &sa, NULL);
fprintf(stderr, "Supervisor ready on %s\n", CONTROL_PATH);
//todo 4
while(!ctx.should_stop){
      int client_fd;
      control_request_t req;
      control_response_t resp;
      client_fd = accept(ctx.server_fd, NULL, NULL);
      if(client_fd < 0){
         if(errno == EINTR){
                  continue;
         }
         break;
      }
      memset(&req, 0, sizeof(req));
      memset(&resp, 0, sizeof(resp));
      if(read(client_fd, &req, sizeof(req))!=sizeof(req)){
         close(client_fd);
         continue;
      }
      switch(req.kind){
         case CMD_START:
         case CMD_RUN:
                  handle_start(&ctx, &req, &resp, client_fd);
                  break;
         case CMD_PS:
                  handle_ps(&ctx, &resp, client_fd);
                  break;
         case CMD_LOGS:
                  handle_logs(&ctx, &req, &resp, client_fd);
                  break;
         case CMD_STOP:
                  handle_stop(&ctx, &req, &resp);
                  write(client_fd, &resp, sizeof(resp));
                  break;
         default:
                  resp.status = -1;
                  snprintf(resp.message, sizeof(resp.message), "unknown command");
                  write(client_fd, &resp, sizeof(resp));

      }
      close(client_fd);

      reap_children(&ctx);
}
//todo 5
bounded_buffer_begin_shutdown(&ctx.log_buffer);
pthread_join(ctx.logger_thread, NULL);
bounded_buffer_destroy(&ctx.log_buffer);
```

```
    pthread_mutex_destroy(&ctx.metadata_lock);
    if(ctx.monitor_fd >= 0) close(ctx.monitor_fd);
    close(ctx.server_fd);
    unlink(CONTROL_PATH);
    return 0;
}
static int send_control_request(const control_request_t *req)
{
    int fd;
    struct sockaddr_un addr;
    control_response_t resp;
    fd = socket(AF_UNIX, SOCK_STREAM, 0);
    if(fd<0){
        perror("socket");
        return 1;
    }
    memset(&addr, 0, sizeof(addr));
    addr.sun_family = AF_UNIX;
    strncpy(addr.sun_path, CONTROL_PATH, sizeof(addr.sun_path) - 1);
    if (connect(fd, (struct sockaddr *)&addr, sizeof(addr)) < 0) {
        perror("connect - is the supervisor running?");
        close(fd);
        return 1;
    }
    if (write(fd, req, sizeof(*req)) != sizeof(*req)) {
        perror("write");
        close(fd);
        return 1;
    }
    if (req->kind == CMD_LOGS) {
        char buf[4096];
        ssize_t n;
        while ((n = read(fd, buf, sizeof(buf))) > 0)
            write(STDOUT_FILENO, buf, n);
        close(fd);
        return 0;
    }
    if (req->kind == CMD_PS) {
        char buf[4096];
        ssize_t n;
        while ((n = read(fd, buf, sizeof(buf))) > 0)
            write(STDOUT_FILENO, buf, n);
        close(fd);
        return 0;
    }
    if (req->kind == CMD_RUN) {
        control_response_t r1, r2;
        if (read(fd, &r1, sizeof(r1)) == sizeof(r1))
            printf("%s\n", r1.message);
        if (read(fd, &r2, sizeof(r2)) == sizeof(r2)) {
            printf("%s\n", r2.message);
            close(fd);
            return r2.status == 0 ? 0 : 1;
        }
        close(fd);
        return 0;
}
    if (read(fd, &resp, sizeof(resp)) == sizeof(resp)) {
        printf("%s\n", resp.message);
        close(fd);
        return resp.status == 0 ? 0 : 1;
    }
    close(fd);
```

```
    return 0;
}
static int cmd_start(int argc, char *argv[])
{
    control_request_t req;
    if (argc < 5) {
        fprintf(stderr,
                "Usage: %s start <id> <container-rootfs> <command> [--soft-mib N] [--hard-mib N] [--nice N]\n",
                argv[0]);
        return 1;
    }
    memset(&req, 0, sizeof(req));
    req.kind = CMD_START;
    strncpy(req.container_id, argv[2], sizeof(req.container_id) - 1);
    strncpy(req.rootfs, argv[3], sizeof(req.rootfs) - 1);
    strncpy(req.command, argv[4], sizeof(req.command) - 1);
    req.soft_limit_bytes = DEFAULT_SOFT_LIMIT;
    req.hard_limit_bytes = DEFAULT_HARD_LIMIT;
    if (parse_optional_flags(&req, argc, argv, 5) != 0)
        return 1;
    return send_control_request(&req);
}
static int cmd_run(int argc, char *argv[])
{
    control_request_t req;
    if (argc < 5) {
        fprintf(stderr,
                "Usage: %s run <id> <container-rootfs> <command> [--soft-mib N] [--hard-mib N] [--nice N]\n",
                argv[0]);
        return 1;
    }
    memset(&req, 0, sizeof(req));
    req.kind = CMD_RUN;
    strncpy(req.container_id, argv[2], sizeof(req.container_id) - 1);
    strncpy(req.rootfs, argv[3], sizeof(req.rootfs) - 1);
    strncpy(req.command, argv[4], sizeof(req.command) - 1);
    req.soft_limit_bytes = DEFAULT_SOFT_LIMIT;
    req.hard_limit_bytes = DEFAULT_HARD_LIMIT;
    if (parse_optional_flags(&req, argc, argv, 5) != 0)
        return 1;
    return send_control_request(&req);
}
static int cmd_ps(void)
{
    control_request_t req;
    memset(&req, 0, sizeof(req));
    req.kind = CMD_PS;
    printf("Expected states include: %s, %s, %s, %s, %s\n",
           state_to_string(CONTAINER_STARTING),
           state_to_string(CONTAINER_RUNNING),
           state_to_string(CONTAINER_STOPPED),
           state_to_string(CONTAINER_KILLED),
           state_to_string(CONTAINER_EXITED));
    return send_control_request(&req);
}
static int cmd_logs(int argc, char *argv[])
{
    control_request_t req;
    if (argc < 3) {
        fprintf(stderr, "Usage: %s logs <id>\n", argv[0]);
        return 1;
    }
    memset(&req, 0, sizeof(req));
```

```
    req.kind = CMD_LOGS;
    strncpy(req.container_id, argv[2], sizeof(req.container_id) - 1);
    return send_control_request(&req);
}
static int cmd_stop(int argc, char *argv[])
{
    control_request_t req;
    if (argc < 3) {
        fprintf(stderr, "Usage: %s stop <id>\n", argv[0]);
        return 1;
    }
    memset(&req, 0, sizeof(req));
    req.kind = CMD_STOP;
    strncpy(req.container_id, argv[2], sizeof(req.container_id) - 1);
    return send_control_request(&req);
}
int main(int argc, char *argv[])
{
    if (argc < 2) {
        usage(argv[0]);
        return 1;
    }
    if (strcmp(argv[1], "supervisor") == 0) {
        if (argc < 3) {
            fprintf(stderr, "Usage: %s supervisor <base-rootfs>\n", argv[0]);
            return 1;
        }
        return run_supervisor(argv[2]);
    }
    if (strcmp(argv[1], "start") == 0)
        return cmd_start(argc, argv);
    if (strcmp(argv[1], "run") == 0)
        return cmd_run(argc, argv);
    if (strcmp(argv[1], "ps") == 0)
        return cmd_ps();
    if (strcmp(argv[1], "logs") == 0)
        return cmd_logs(argc, argv);
    if (strcmp(argv[1], "stop") == 0)

        return cmd_stop(argc, argv);
    usage(argv[0]);
    return 1;
}
```

monitor.c - Multi-Container Memory Monitor (Linux Kernel Module)

```
/** monitor.c - Multi-Container Memory Monitor (Linux Kernel Module)*/
#include <linux/cdev.h>
#include <linux/device.h>
#include <linux/fs.h>
#include <linux/kernel.h>
#include <linux/list.h>
#include <linux/mm.h>
#include <linux/module.h>
#include <linux/mutex.h>
#include <linux/pid.h>
#include <linux/sched/signal.h>
#include <linux/slab.h>
#include <linux/timer.h>
#include <linux/uaccess.h>
#include <linux/version.h>
#include "monitor_ioctl.h"
```

```
#define DEVICE_NAME "container_monitor"
#define CHECK_INTERVAL_SEC 1
struct monitor_entry {
    pid_t pid;
    char container_id[32];
    unsigned long soft_limit;
    unsigned long hard_limit;
    int soft_triggered;
    struct list_head list;
};
static LIST_HEAD(monitor_list);
static DEFINE_MUTEX(monitor_lock);
/* --- Provided: internal device / timer state --- */
static struct timer_list monitor_timer;
static dev_t dev_num;
static struct cdev c_dev;
static struct class *cl;
static long get_rss_bytes(pid_t pid)
{
    struct task_struct *task;
    struct mm_struct *mm;
    long rss_pages = 0;
    rcu_read_lock();
    task = pid_task(find_vpid(pid), PIDTYPE_PID);
    if (!task) {
        rcu_read_unlock();
        return -1;
    }
    get_task_struct(task);
    rcu_read_unlock();
    mm = get_task_mm(task);
    if (mm) {
        rss_pages = get_mm_rss(mm);
        mmput(mm);
    }
    put_task_struct(task);
    return rss_pages * PAGE_SIZE;
}
static void log_soft_limit_event(const char *container_id,
                                 pid_t pid,
                                 unsigned long limit_bytes,
                                 long rss_bytes)
{
    printk(KERN_WARNING
           "[container_monitor] SOFT LIMIT container=%s pid=%d rss=%ld limit=%lu\n",
           container_id, pid, rss_bytes, limit_bytes);
}
static void kill_process(const char *container_id,
                         pid_t pid,
                         unsigned long limit_bytes,
                         long rss_bytes)
{
    struct task_struct *task;
    rcu_read_lock();
    task = pid_task(find_vpid(pid), PIDTYPE_PID);
    if (task)
        send_sig(SIGKILL, task, 1);
    rcu_read_unlock();
    printk(KERN_WARNING
           "[container_monitor] HARD LIMIT container=%s pid=%d rss=%ld limit=%lu\n",
           container_id, pid, rss_bytes, limit_bytes);
}
static void timer_callback(struct timer_list *t)
```

```
{
    struct monitor_entry *entry, *tmp;
    mutex_lock(&monitor_lock);
     list_for_each_entry_safe(entry, tmp, &monitor_list, list) {
    long rss = get_rss_bytes(entry->pid);
    /* 1. Remove dead process */
    if (rss < 0) {
        list_del(&entry->list);
        kfree(entry);
        continue;
    }
    /* 2. HARD LIMIT */
    if (rss > entry->hard_limit) {
        kill_process(entry->container_id,
                     entry->pid,
                     entry->hard_limit,
                     rss);
        list_del(&entry->list);
        kfree(entry);
        continue;
    }
    /* 3. SOFT LIMIT (only once) */
    if (rss > entry->soft_limit && !entry->soft_triggered) {
        log_soft_limit_event(entry->container_id,
                             entry->pid,
                             entry->soft_limit,
                             rss);
        entry->soft_triggered = 1;
    }
}
mutex_unlock(&monitor_lock);
    mod_timer(&monitor_timer, jiffies + CHECK_INTERVAL_SEC * HZ);
}
static long monitor_ioctl(struct file *f, unsigned int cmd, unsigned long arg)
{
    printk(KERN_INFO "[container_monitor] IOCTL HIT\n");
    struct monitor_request req;
    (void)f;
    if (cmd != MONITOR_REGISTER && cmd != MONITOR_UNREGISTER)
        return -EINVAL;
    if (copy_from_user(&req, (struct monitor_request __user *)arg, sizeof(req)))
        return -EFAULT;
    if (cmd == MONITOR_REGISTER) {
        printk(KERN_INFO
               "[container_monitor] Registering container=%s pid=%d soft=%lu hard=%lu\n",
               req.container_id, req.pid, req.soft_limit_bytes, req.hard_limit_bytes);
struct monitor_entry *entry;
if (req.soft_limit_bytes > req.hard_limit_bytes)
    return -EINVAL;
entry = kmalloc(sizeof(*entry), GFP_KERNEL);
if (!entry)
    return -ENOMEM;
entry->pid = req.pid;
strncpy(entry->container_id, req.container_id,
        sizeof(entry->container_id) - 1);
entry->container_id[sizeof(entry->container_id) - 1] = '\0';
entry->soft_limit = req.soft_limit_bytes;
entry->hard_limit = req.hard_limit_bytes;
entry->soft_triggered = 0;
mutex_lock(&monitor_lock);
list_add(&entry->list, &monitor_list);
mutex_unlock(&monitor_lock);
        return 0;
```

```
    }
    printk(KERN_INFO
           "[container_monitor] Unregister request container=%s pid=%d\n",
           req.container_id, req.pid);
struct monitor_entry *entry, *tmp;
mutex_lock(&monitor_lock);
list_for_each_entry_safe(entry, tmp, &monitor_list, list) {
    if (entry->pid == req.pid) {
        list_del(&entry->list);
        kfree(entry);
        mutex_unlock(&monitor_lock);
        return 0;
    }
}
mutex_unlock(&monitor_lock);
return -ENOENT;
}
/* --- Provided: file operations --- */
static struct file_operations fops = {
    .owner = THIS_MODULE,
    .unlocked_ioctl = monitor_ioctl,
};
/* --- Provided: Module Init --- */
static int __init monitor_init(void)
{
    if (alloc_chrdev_region(&dev_num, 0, 1, DEVICE_NAME) < 0)
        return -1;
#if LINUX_VERSION_CODE >= KERNEL_VERSION(6, 4, 0)
    cl = class_create(DEVICE_NAME);
#else
    cl = class_create(THIS_MODULE, DEVICE_NAME);
#endif
    if (IS_ERR(cl)) {
        unregister_chrdev_region(dev_num, 1);
        return PTR_ERR(cl);
    }
    if (IS_ERR(device_create(cl, NULL, dev_num, NULL, DEVICE_NAME))) {
        class_destroy(cl);
        unregister_chrdev_region(dev_num, 1);
        return -1;
    }
    cdev_init(&c_dev, &fops);
    if (cdev_add(&c_dev, dev_num, 1) < 0) {
        device_destroy(cl, dev_num);
        class_destroy(cl);
        unregister_chrdev_region(dev_num, 1);
        return -1;
    }
    timer_setup(&monitor_timer, timer_callback, 0);
    mod_timer(&monitor_timer, jiffies + CHECK_INTERVAL_SEC * HZ);
    printk(KERN_INFO "[container_monitor] Module loaded. Device: /dev/%s\n", DEVICE_NAME);
    return 0;
}
/* --- Provided: Module Exit --- */
static void __exit monitor_exit(void)
{
    del_timer_sync(&monitor_timer);
struct monitor_entry *entry, *tmp;
mutex_lock(&monitor_lock);
list_for_each_entry_safe(entry, tmp, &monitor_list, list) {
    list_del(&entry->list);
    kfree(entry);
}
```

```
mutex_unlock(&monitor_lock);
    cdev_del(&c_dev);
    device_destroy(cl, dev_num);
    class_destroy(cl);
    unregister_chrdev_region(dev_num, 1);
    printk(KERN_INFO "[container_monitor] Module unloaded.\n");
}
module_init(monitor_init);
module_exit(monitor_exit);
MODULE_LICENSE("GPL");
MODULE_DESCRIPTION("Supervised multi-container memory monitor");
```

CHAPTER 5:- RESULT AND DISCUSSION

5.1 Overview

The system was tested under different scenarios to evaluate its behavior under concurrent execution, varying workloads, and resource constraints. The focus of testing was not just correctness, but also how the system behaves under real operating conditions.

5.2 Behavior Under Multiple Containers

When multiple containers were executed simultaneously, it was observed that all containers continued to run independently without interference.

Even when one container was stopped or terminated, the others continued execution without any disruption. This indicates that process isolation and supervision were functioning correctly.

5.3 Logging Behavior Under Concurrency

During execution, multiple containers generated output at the same time. The logging system was able to capture all outputs without mixing or losing data.

An important observation was that even when containers terminated abruptly, their remaining output was still written to log files. This shows that the logging pipeline handled concurrency and shutdown conditions effectively.

5.4 System Response to User Commands

Commands such as start, stop, and logs were executed while containers were running.

The system responded correctly without delays or inconsistencies. Containers could be controlled dynamically without restarting the supervisor, showing that the control mechanism is stable under runtime conditions.

5.5 Memory Limit Behavior

When containers were executed with memory-intensive workloads, different behaviors were observed based on configured limits:

- Crossing the soft limit generated warning messages without stopping execution
- Crossing the hard limit resulted in immediate termination of the container

This demonstrates that memory control is enforced strictly and behaves as expected under stress conditions.

5.6 Scheduling Observations

When containers were run with different priority levels, a noticeable difference in execution speed was observed.

Higher priority containers completed their tasks faster, while lower priority ones took more time. This confirms that CPU scheduling is influenced by priority levels and that the system correctly reflects Linux scheduling behavior.

5.7 System Stability and Cleanup

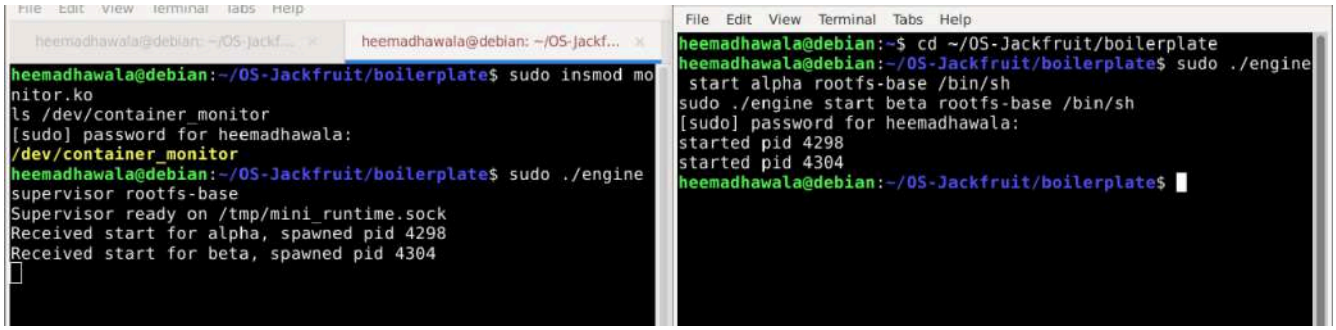
After stopping all containers, the system was checked for leftover processes and resources.

No zombie processes were found, and all resources were released properly. This indicates that process cleanup and shutdown handling are correctly implemented.

CHAPTER 6:- OUTPUTS

6.1 Multi-container supervision

Description:Two containers (alpha, beta) running simultaneously under one supervisor process



COMMANDS

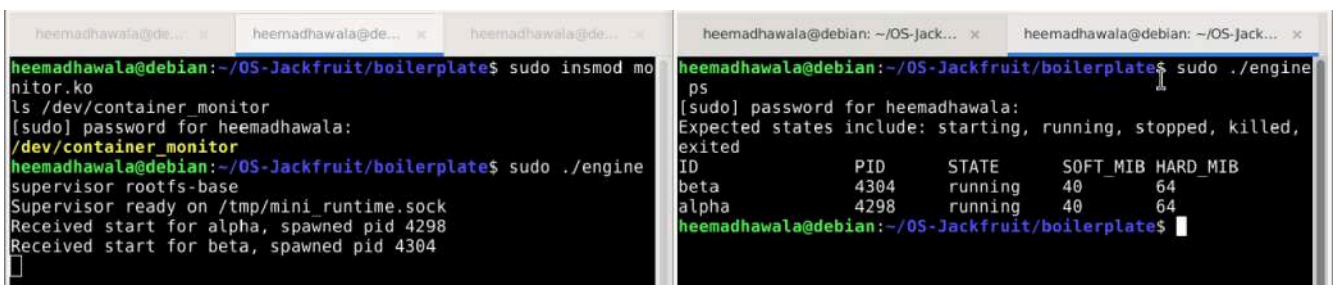
sudo ./engine supervisor rootfs-base (TERMINAL -1)

sudo ./engine start alpha rootfs-base /bin/sh (TERMINAL-2)

sudo ./engine start beta rootfs-base /bin/sh(TERMINAL-2)

6.2 Metadata tracking

Description:Output of engine ps showing container ID, PID, state, and memory limits.

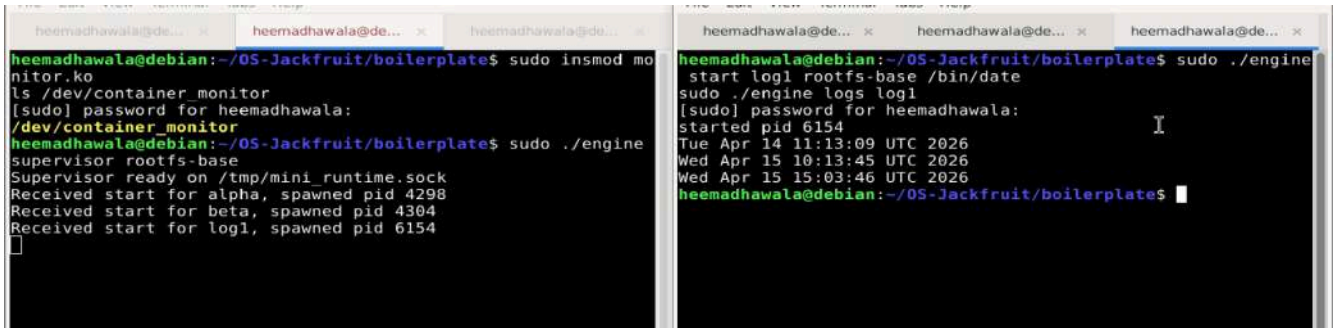


COMMANDS

sudo ./engine ps

6.3 Bounded-buffer logging

Description:Log output captured through the producer–consumer logging pipeline.



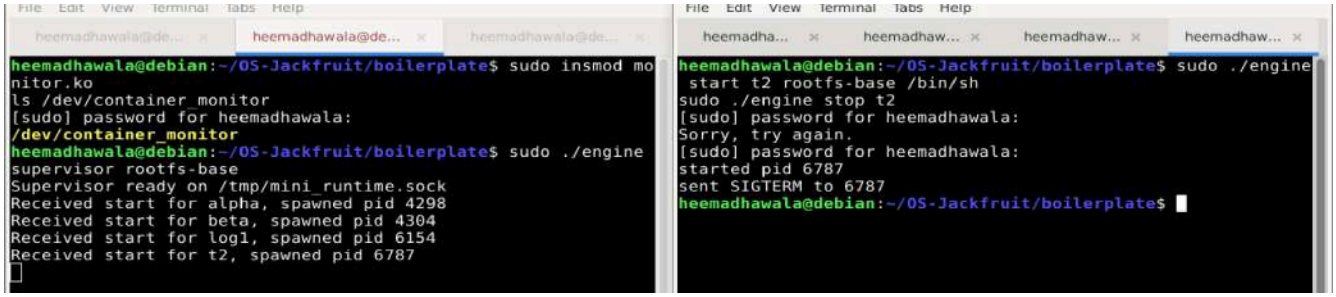
COMMANDS

sudo ./engine start log1 rootfs-base /bin/date

sudo ./engine logs log1

6.4 CLI and IPC

Description:A CLI stop command sent to the supervisor via UNIX socket and handled correctly.



COMMANDS

sudo ./engine start t2 rootfs-base /bin/sh

sudo ./engine stop t2

6.5 Soft-limit warning

Description:Kernel logs showing warning when container exceeds soft memory limit.

```
heemadhawala@debian:~/OS-Jackfruit/boilerplate$ sudo ./engine supervisor rootfs-base
Supervisor ready on /tmp/mini_runtime.sock
Received start for alpha, spawned pid 4298
Received start for beta, spawned pid 4304
Received start for log1, spawned pid 6154
Received start for t2, spawned pid 6787
Received start for soft, spawned pid 7387
Received start for soft, spawned pid 8576
Received start for high, spawned pid 9229
Received start for low, spawned pid 9235
Received start for soft, spawned pid 9942
Received start for soft, spawned pid 10034
```

```
heemadhawala@debian:~/OS-Jackfruit/boilerplate$ sudo dmesg -C
cp memory_hog rootfs-base/

sudo ./engine start soft rootfs-base ./memory_hog
sleep 6
sudo dmesg | grep container_monitor
started pid 10034
[ 1206.734617] [container_monitor] IOCTL HIT
[ 1206.734644] [container_monitor] Registering container=soft pid=10034 soft=41943040 hard=67108864
[ 1210.988649] [container_monitor] SOFT LIMIT container=soft pid=10034 rss=42422272 limit=41943040
```

COMMANDS

sudo ./engine start soft rootfs-base ./memory_hog

sudo dmesg | grep container_monitor

6.6 Hard-limit enforcement

Description:Kernel logs showing process termination when hard memory limit is exceeded.

```
heemadhawala@debian:~/OS-Jackfruit/boilerplate$ sudo ./engine supervisor rootfs-base
Supervisor ready on /tmp/mini_runtime.sock
Received start for alpha, spawned pid 4298
Received start for beta, spawned pid 4304
Received start for log1, spawned pid 6154
Received start for t2, spawned pid 6787
Received start for soft, spawned pid 7387
Received start for soft, spawned pid 8576
Received start for high, spawned pid 9229
Received start for low, spawned pid 9235
Received start for soft, spawned pid 9942
Received start for soft, spawned pid 10034
Received start for hard, spawned pid 10117
Received start for hard1, spawned pid 10740
Received start for hard2, spawned pid 10746
```

```
heemadhawala@debian:~/OS-Jackfruit/boilerplate$ sudo ./engine start hard1 rootfs-base ./memory_hog
sudo ./engine start hard2 rootfs-base ./memory_hog
[sudo] password for heemadhawala:
started pid 10740
started pid 10746
heemadhawala@debian:~/OS-Jackfruit/boilerplate$ sudo dmesg | grep "HARD LIMIT"
[ 1273.484112] [container_monitor] HARD LIMIT container=hard pid=10117 rss=67674112 limit=67108864
```

6.7 Scheduling experiment

Description:Two containers running CPU workloads with different priorities showing scheduling behavior.

```
heemadhawala@debian:~/OS-Jackfruit/boilerplate$ sudo ./engine
supervisor rootfs-base
Supervisor ready on /tmp/mini_runtime.sock
Received start for alpha, spawned pid 4298
Received start for beta, spawned pid 4304
Received start for log1, spawned pid 6154
Received start for t2, spawned pid 6787
Received start for soft, spawned pid 7387
Received start for soft, spawned pid 8576
Received start for high, spawned pid 9229
Received start for low, spawned pid 9235
```

```
Applications  Xfce4-terminal                                   2026-04-15 20:43   HEEMADHAWALA R
Terminal - heemadhawala@debian: ~/OS-Jackfruit/boilerplate
File  Edit  View  Terminal  Tabs  Help
heemadhaw...  heemadhaw...  heemadhaw...  heemadhaw...  heemadhaw...  heemadhaw...  heemadhaw...  heemadhaw...
heemadhawala@debian:~/OS-Jackfruit/boilerplate$ cp cpu_hog rootfs-base/
heemadhawala@debian:~/OS-Jackfruit/boilerplate$ sudo ./engine start high rootfs-base ./cpu_hog --nice -5
sudo ./engine start low rootfs-base ./cpu_hog --nice 10
[sudo] password for heemadhawala:
started pid 9229
started pid 9235
heemadhawala@debian:~/OS-Jackfruit/boilerplate$ sudo ./engine logs high
sudo ./engine logs low
cpu_hog alive elapsed=1 accumulator=1174336596569951113
cpu_hog alive elapsed=2 accumulator=1270570694773282540
cpu_hog alive elapsed=3 accumulator=6536592348963765490
cpu_hog alive elapsed=4 accumulator=2748803756655716212
cpu_hog alive elapsed=5 accumulator=5471689385478181812
cpu_hog alive elapsed=6 accumulator=1879574134951606724
cpu_hog alive elapsed=7 accumulator=1643961655224131359 9
cpu_hog alive elapsed=8 accumulator=1498557797482487024 1
cpu_hog alive elapsed=9 accumulator=1170142964692501312 2
cpu_hog done duration=10 accumulator=1239747645398993100 3
execv: No such file or directory
execv: No such file or directory
cpu_hog alive elapsed=1 accumulator=1233590843291185639 4
cpu_hog alive elapsed=2 accumulator=2596920147878098 4
cpu_hog alive elapsed=3 accumulator=1353745438591069812 5
cpu_hog alive elapsed=4 accumulator=8601639872802421761
cpu_hog alive elapsed=5 accumulator=5173082558239548135
cpu_hog alive elapsed=6 accumulator=8001091139535629
cpu_hog alive elapsed=7 accumulator=1496811406169781300
cpu_hog alive elapsed=8 accumulator=1610235668419644327 7
cpu_hog alive elapsed=9 accumulator=1783813605754300881 4
cpu_hog done duration=10 accumulator=7847746745033548417
cpu_hog alive elapsed=1 accumulator=3853890695891366752
cpu_hog alive elapsed=2 accumulator=2052502631246739177
cpu_hog alive elapsed=3 accumulator=3747542254937611419
cpu_hog alive elapsed=4 accumulator=3569131114005427723
cpu_hog alive elapsed=5 accumulator=1426253423719081881 0
cpu_hog alive elapsed=6 accumulator=1350300812015815136 3
cpu_hog alive elapsed=7 accumulator=1522619975596325247 9
cpu_hog alive elapsed=8 accumulator=1200427456992644821
cpu_hog alive elapsed=9 accumulator=1050986737217242722 8
cpu_hog alive elapsed=10 accumulator=1467938337008198750 8
cpu_hog done duration=10 accumulator=1467938337008198750 8
execv: No such file or directory
```

COMMANDS

sudo ./engine start high rootfs-base ./cpu_hog --nice -5
sudo ./engine start low rootfs-base ./cpu_hog --nice 10
sudo ./engine logs high
sudo ./engine logs low

6.8Clean teardown

Description:All processes stopped and kernel module unloaded cleanly.

```
heemadhawala@debian:~/OS-Jackfruit/boilerplate$ sudo pkill engine
ps aux | grep engine

sudo rmmod monitor
sudo dmesg | tail
heemadh+   11483  0.0  0.0   6252  2008 pts/13   S+   20:49   0:00 grep engine
[ 1273.484112] [container_monitor] HARD LIMIT container=hard pid=10117 rss=67674112 limit=67108864
[ 1323.782976] [container_monitor] IOCTL HIT
[ 1323.783009] [container_monitor] Registering container=hard1 pid=10740 soft=41943040 hard=67108864
[ 1323.797328] [container_monitor] IOCTL HIT
[ 1323.797331] [container_monitor] Registering container=hard2 pid=10746 soft=41943040 hard=67108864
[ 1328.782082] [container_monitor] SOFT LIMIT container=hard2 pid=10746 rss=42520576 limit=41943040
[ 1328.782146] [container_monitor] SOFT LIMIT container=hard1 pid=10740 rss=42516480 limit=41943040
[ 1331.853833] [container_monitor] HARD LIMIT container=hard2 pid=10746 rss=67579904 limit=67108864
[ 1331.853981] [container_monitor] HARD LIMIT container=hard1 pid=10740 rss=67706880 limit=67108864
[ 1485.223461] [container_monitor] Module unloaded.
```

COMMANDS

sudo pkill engine

ps aux | grep engine

sudo rmmod monitor

sudo dmesg | tail

CHAPTER 7 :- GITHUB LINK

The complete source code for this project is available at:
https://github.com/Hema3156/OS-Jackfruit/tree/main
https://github.com/Heemadhawala-R/OS-Jackfruit

The repository contains:

- boilerplate/engine.c — User-space runtime and supervisor
- boilerplate/monitor.c — Linux Kernel Module for memory monitoring
- boilerplate/monitor_ioctl.h — Shared ioctl definitions
- boilerplate/cpu_hog.c — CPU-bound test workload
- boilerplate/io_pulse.c — I/O-bound test workload
- boilerplate/memory_hog.c — Memory-consuming test workload
- boilerplate/Makefile — Build targets for all components
- boilerplate/environment-check.sh — VM environment preflight check
- README.md — Build instructions, demo screenshots, engineering analysis
- project-guide.md — Full project specification

REFERENCES /BIBLIOGRAPHY

1)Silberschatz, A., Galvin, P. B., & Gagne, G. (2018). Operating System Concepts, 10th Edition. Wiley.

2)Docker Inc. (2024). Understand container isolation. https://docs.docker.com/get-started/overview/

3)OS-Jackfruit Project Repository - https://github.com/shivangjhalani/OS-Jackfruit

APPENDIX A DEFINITIONS, ACRONYMS AND ABBREVIATIONS

Term	Definition
CFS	Completely Fair Scheduler — the default Linux process scheduler that allocates CPU time proportionally by process weight.
CLI	Command-Line Interface — a text-based user interface for interacting with software.
clone()	Linux system call that creates a new process and optionally shares specific namespaces or resources with the parent.
chroot()	System call that changes the apparent root directory for the current process and its children.
container_record_t	User-space C struct tracking per-container metadata: ID, PID, state, limits, log path, and exit information.
IPC	Inter-Process Communication — mechanisms for processes to exchange data (e.g., pipes, sockets, shared memory).
ioctl	Input/Output Control — a system call used to perform device-specific operations not covered by standard read/write.
LKM	Linux Kernel Module — a piece of code that can be dynamically

	loaded and unloaded into the kernel.
monitor_entry	Kernel-space struct tracking a container PID and its soft/hard memory limits in the kernel linked list.
Namespace	A Linux kernel feature that isolates a specific resource view (PID, UTS, mount, network, etc.) per group of processes.
nice value	An integer (-20 to +19) that adjusts a process's scheduling priority. Lower values mean higher priority.
PID	Process IDentifier — a unique integer assigned by the kernel to each process.
pivot_root()	System call that changes the root mount of the calling process; more secure than chroot for container use.
POSIX	Portable Operating System Interface — a family of standards for maintaining compatibility between operating systems.
RSS	Resident Set Size — the portion of a process's memory that is held in physical RAM.
SIGCHLD	Signal sent to a parent process when a child process terminates, is stopped, or is resumed.

SIGKILL	An unblockable signal that immediately terminates a process.
SIGTERM	A signal requesting graceful termination of a process (can be caught and handled).
UNIX socket	A socket that communicates between processes on the same host using a filesystem pathname, not a network address.
UTS namespace	Unix Time-sharing System namespace — isolates hostname and NIS domain name per process group.
waitpid()	System call used by a parent process to wait for state changes in a child process and collect its exit status.